Duitlah — Bagilah & Groups test pass

Tester: Muhammad Haikal **Date:** 2 September 2026 **Build:** `main`, commit of 24 Aug 2026 (confirmed current)
Environment: Android emulator (Pixel 8, API 36) · local Docker stack · web companion on `localhost:3001` · `ANTHROPIC_API_KEY` unset

Executive summary

The calculation engines are correct. The screens that drive them are incomplete.

Every arithmetic result tested came out exact, including the hard cases — RM 25.00 split three ways resolved to 8.34 / 8.33 / 8.33 by largest remainder, and an RM 85.55 group expense split to shares summing to the sen. The allocation logic, the settlement generator and the server-side validation guards all behave as SYSTEM_LOGIC_AND_FEATURES.md describes.

What is missing is the interface. Three separate repository methods are fully implemented, wired to live API endpoints, and called by nothing. The consequence is that **the Bagilah flow cannot be completed**: a bill cannot be finalized, because finalization requires payment data, and no screen exists to enter it. Tax, service charge and the printed receipt total likewise have no input anywhere, so a manually created bill can only ever split its raw item subtotal — an incomplete figure for essentially every Malaysian restaurant bill.

The Groups path is wired end to end and was tested successfully through to a settlement obligation, which is how the settlement engine could be verified at all.

16 issues found: 1 blocker, 2 critical, 4 high, 4 medium, 5 low. 8 checks passed.

Priority order

#	Issue	Severity
B1	Bill cannot be finalized — no UI to record who paid	**Blocker**
C1	Tax, service charge and receipt total have no UI	**Critical**
C2	"Add participant" button unreachable behind the keyboard	**Critical**
H1	Bagilah invite link uses wrong port and bare `localhost`	High
H2	Group invite links point at the Bagilah join route	High
H3	`/groups/join?token=…` rejects valid tokens	High
H4	"You're owed / You owe" shows stale zeros until refreshed	High

The recurring pattern

Three of the most serious findings share one shape: **a repository method exists, its endpoint is live, and no screen calls it.**

Method	Endpoint	UI callers
`bagilahRepository.recordPayment`	`PUT /api/v1/bills/{id}/payments/{participantId}`	none
`bagilahRepository.removePayment`	`DELETE /api/v1/bills/{id}/payments/{participantId}`	none
`settlementsRepository.cancel`	`POST …/cancel`	none
`settlementsRepository.dispute`	`POST …/dispute`	none

Plus `declaredTotal`, `roundingAdjustment` and the tax/service/discount fields, which exist in the domain model and the `PATCH /api/v1/bills/{id}` payload but have no inputs outside the receipt-review screen.

Worth auditing every repository for orphaned methods before treating a feature as done — the backend consistently runs ahead of the client, which makes the app look less finished than it is.

What passed

1. Registration, login, session restore. Clean, no re-onboarding on return.

2. Unclaimed-item guard. Finalize correctly disabled: *"3 item(s) still need claiming before this bill can be finalized."*

3. The Bagilah allocation engine is exact. Items: 3 × Nasi Lemak Ayam @ 12.90, 2 × Teh Ais @ 3.50, 1 × Sotong Goreng @ 25.00. Claims: Haikal 2 nasi + ⅓ sotong; Aiman 1 nasi + 1 teh + ⅓ sotong; Siti 1 teh + ⅓ sotong.

Person	Expected	Displayed
muhammad haikal	25.80 + 8.34 = 34.14	**34.14** ✓
aiman	12.90 + 3.50 + 8.33 = 24.73	**24.73** ✓
siti	3.50 + 8.33 = 11.83	**11.83** ✓
Total	**70.70**	**70.70** ✓

RM 25.00 across three people resolved to **8.34 / 8.33 / 8.33** — largest-remainder allocation exactly as specified in §10.E. No sen lost, none invented.

4. Guest join via invite link. Both guests joined without accounts, were labelled `(guest)`, and could view and claim items.

5. SSE live updates. Guests joining on the web appeared on the owner's mobile screen with no refresh.

6. All four claim types present. Whole item, split equally, and Custom offering %, Qty and RM — matching §10.D.

7. Group expense splitting is exact. RM 85.55 split equally between two members produced **+42.77 / −42.77**, netting to zero. 85.55 ÷ 2 = 42.775, so the payer carries 42.78 and the shares sum to precisely 85.55.

8. Group settlement generation works. "Settle up" produced a correct obligation — `aiman → muhammad haikal · RM 42.77 · UNPAID` — and zeroed the group balances. This is the settlement path Bagilah cannot currently reach, so

the settlement engine is verified by this route.

Blocker

B1 — A Bagilah bill cannot be finalized; no UI records who paid

Where: `features/bagilah/data/bagilah_repository.dart:159-165` vs `presentation/`

Tapping "Finalize bill" returns **"Record who paid before finalizing."** The server correctly refuses to finalize without payment data. No interface anywhere in the app can supply it.

`recordPayment()` and `removePayment()` are implemented in the repository. A grep across the whole of mobile `lib/` finds no caller for either — the only `_recordPayment` handlers belong to `features/business/` and `features/liabilities/`, unrelated features. `bill_detail_screen.dart:346` renders `'Paid RM ${b.paid} · Owed RM ${b.owed}'`, so the display side anticipates payment data that nothing can write.

§10.F: *"Before finalization, the owner records how much each participant actually paid at the merchant."* That step has no screen.

Consequence: the flagship flow cannot be completed. Bills remain permanently in DRAFT. No settlements are ever generated from Bagilah, and everything downstream — settlement obligations, debt simplification, payment handoff, finalized history — is unreachable.

Positive note: the backend invariant is correctly enforced and the message is clear, specific, human English. The validation layer works. Only the screen is absent.

Critical

C1 — Tax, service charge and the actual receipt total have no UI

Where: `create_bill_screen.dart`, `bill_detail_screen.dart`, `receipt_review_screen.dart`

PRODUCT.md's core promise: *"an exact split that reconciles to the cent against the real receipt total — including tax, service charge, discounts, and rounding."* That reconciliation cannot be performed from the app.

The backend supports it fully. `domain/bill.dart` carries `declaredTotal`, `total`, `roundingAdjustment`, `subtotal`, `computedTotal` and per-participant `roundingShare`. The repository exposes `updateBill` (`PATCH /api/v1/bills/{id}`).

The UI does not.

Screen	Reached via	Charge fields
`create_bill_screen.dart`	"I'll enter it manually"	Merchant, date. Nothing else.
`receipt_review_screen.dart`	scan / gallery only	Tax, Service, Discount. **No receipt total.**
`bill_detail_screen.dart`	after creation	**Zero references** to any of them (verified by grep)

Three consequences:

1. A manually created bill can never carry tax or service charge — only the raw item subtotal. A typical Malaysian restaurant bill with 10% service and 6% SST is understated by roughly 16%.
2. Charges cannot be edited after creation. Set one wrong and the only remedy is to delete the bill and re-enter it through the scan flow.
3. `declaredTotal` has no input anywhere. The rounding-adjustment mechanism — the specific thing that makes a Duitlah split reconcile against a printed receipt, and the clearest differentiator from any other splitting app — cannot be reached by a user.

Observed: a bill of RM 70.70 in items offered an enabled "Finalize bill" having never asked for tax, service charge or a receipt total.

This is a UI completeness gap, not an engine defect. The calculation layer is correct (Pass 3). The work is exposing what already exists: charge fields on manual creation, an editable charges section on the bill screen, and a receipt-total input feeding `declaredTotal`.

C2 — "Add" unreachable in the Add participant sheet — no workaround

Where: Bagilah → bill → add participant bottom sheet

The sheet does not resize for the on-screen keyboard. The primary "Add" button is clipped below the visible viewport and cannot be tapped. Every Bagilah bill needs participants, so this sits on the critical path.

Steps: create a bill → tap the blank avatar circle → type a name → attempt to tap Add.

All workarounds failed: back button to dismiss the keyboard (no effect); Enter/Done on the keyboard (does not submit); dragging the sheet handle upward (already at maximum height, drags down only).

Named participants cannot be added at all through this route. Both test participants had to be brought in via the guest invite link instead.

Note: the invite sheet and the claim sheet both render fully with their buttons visible, so this is specific to this sheet rather than a global layout problem. Likely worse on physical devices with gesture navigation, where the system inset is larger.

High

H1 — Bagilah invite link uses the wrong port and a bare `localhost`

Where: `core/config/env.dart`, `presentation/qr_invite_sheet.dart:23`

The invite sheet produced verbatim:

```
http://localhost:3000/bagilah/join?token=WVwi1JYE-dEy2wZTlfwYNZMIGDWnl39pIlnGiS3pJAA
```

Two defects. Port 3000 serves nothing — compose publishes web on **3001**, and `CORS_ALLOWED_ORIGINS` is set to 3001. And `localhost` on a phone resolves to that phone, so every recipient opening the link reaches nothing on their own device.

`Env.apiBaseUrl` correctly special-cases Android with a `10.0.2.2` fallback. `Env.webBaseUrl` has no equivalent. The sheet meanwhile tells the user "Anyone with this link or QR code can join — no account needed", which is currently untrue for every recipient.

Fix: give `webBaseUrl` the same platform-aware treatment as `apiBaseUrl`, and align the dev default with the compose port.

H2 — Group invite links point at the Bagilah join route

Where: `groups/presentation/group_detail_screen.dart:14,35` and `bagilah/presentation/qr_invite_sheet.dart:23`

`group_detail_screen` reuses the Bagilah `QrInviteSheet` for group invitations, passing a group token — but the sheet hardcodes the Bagilah path:

```
String get _link => '${Env.webBaseUrl}/bagilah/join?token=$token';
```

The web app has a dedicated `/groups/join` route that the mobile app never links to. Inviting to the group "Makan Trip" produced `…/bagilah/join?token=…`.

Secondary: the sheet's copy ("no account needed") is true of bills and false of groups, which require registered members. Reusing the component reuses its Bagilah wording.

Fix: parameterise path and copy on `QrInviteSheet` rather than hardcoding.

H3 — `/groups/join?token=…` rejects valid tokens

Where: `apps/web/src/app/groups/join/join-status.tsx`

Confirmed by contradiction: token `z_6PW9hu6MRpBvk6VjrIwQdMhs6SuQjROP5Ybph8NLA` was rejected with "This invite link is invalid or has expired", then **accepted immediately afterwards** by the manual "Join group" dialog on `/groups` — same token, same session, same browser. Three fresh tokens were rejected by the URL route.

Likely cause: the page joins on mount —

```
useEffect(() => { if (token) joinMutation.mutate(); }, [token]);
```

— firing before the auth session hydrates, so the request goes out unauthenticated. The manual dialog runs after hydration and succeeds. Backend join is a plain hash lookup with no expiry, so nothing server-side explains it.

Also: the error message blames the invite link for what appears to be a client-side timing problem, sending users to request a replacement that will fail identically.

Combined with H2: group invites are broken twice over. The only working route into a group is the manual dialog on the web, which the mobile app never mentions.

H4 — "You're owed / You owe" shows stale zeros until manually refreshed

Where: `settlements/application/settlements_providers.dart`, `presentation/settlements_screen.dart`, home dashboard

After settling the group, the Settlements screen showed **"You're owed: RM 0.00"** in the summary tiles while listing **"aiman → muhammad haikal · RM 42.77 remaining · UNPAID"** directly beneath. Two contradictory figures on one

screen.

Pull-to-refresh corrected the tiles to RM 42.77, confirming cache invalidation rather than miscalculation — `settleGroup()` does not invalidate `settlementSummaryProvider`.

Backend verified correct by inspection: `getSummary` nets by counterparty and signs `toUserId == userId` positive; list and summary use the same `findAllInvolvingUser` query; the mobile model matches the response record field for field.

Scope: the same cached summary feeds the home dashboard's cards. Both screens read RM 0.00 until one refresh corrected both. The window is bounded, but inside it the app states a wrong balance confidently and with no staleness indicator — a user who is owed money can be shown "You're owed RM 0.00" on opening the app.

Fix: invalidate the summary and group-balance providers after `settleGroup`, `recordPayment` and bill finalization.

Medium

M1 — Currency renders with one decimal place ("RM 25.0")

The claim sheet header shows `Qty 1.0 · RM 25.0` and `Qty 3.0 · RM 38.7`. Money must always show two decimals. The item list on the same screen formats correctly (`RM 25.00`), so the formatter is applied inconsistently rather than missing. The web client renders correctly, so this is mobile-only. Cosmetic in mechanism, but in a product whose credibility rests on numeric precision it undermines confidence in every other figure on screen.

M2 — No subtotal displayed on the bill screen

Items show line totals (38.70, 7.00, 25.00) with no sum anywhere. The user must add them mentally to reach 70.70. PRODUCT.md commits to "calculations are never a black box" and to users always being able to "see why they owe what they owe"; the most basic intermediate figure is absent.

M3 — Unparseable amount makes "Add expense" silently inert

Entering `RM 85.55` in the group expense Amount field — the label reads "Amount (RM)", so the prefix is an easy mistake — leaves the button completely unresponsive. No validation message, no error, no disabled state, no feedback. `85.55` works immediately. Silent failure is worse than rejection: the user cannot tell whether the app is broken, slow, or refusing their input.

M4 — Settlement cancel and dispute have no UI

§11 states either party can view a settlement, record a payment, **cancel it, or dispute it**. `recordPayment` is wired to the settlements screen; `cancel()` and `dispute()` exist in the repository with no callers anywhere in `lib/`.

Low

L1 — Quantities render as floats

`Qty 3.0`, `Qty 2.0`, `Qty 1.0` in the bill items list. Should read `Qty 3`. The web client formats these correctly as `× 3`.

L2 — Manual bill creation is hidden inside the scan flow

The Bagilah empty state reads "Scan a receipt **or start a bill** to split with friends", presenting two equal paths, but offers only a "Scan receipt" button. Manual entry exists solely as a tertiary text link ("I'll enter it manually") one level down, inside a screen titled "Scan a receipt". Users splitting a bill they did not photograph have no obvious route.

L3 — "Add participant" is an unlabelled blank circle

The control is an empty grey disc beside the owner's avatar — no `+`, no label, no tooltip. Discoverable only by tapping an unexplained shape.

L4 — Inconsistent unclaimed-items copy between clients

Mobile: "3 item(s) still need claiming before this bill can be finalized." Web: "Some items still need claiming before this bill can be finalized." Same state, and the mobile wording is the more useful of the two.

L5 — SSE emitters time out every 30s and flood the API log

`AsyncRequestTimeoutException` stack traces fill `docker compose logs api` continuously as SSE connections hit Spring's default async timeout, making the log very hard to read — which matters most when diagnosing something else, as during this pass. Suggest an explicit `SseEmitter` timeout with quiet completion handling.

Observation — brand inconsistency between clients

Not a defect, but worth a deliberate decision rather than drift.

The mobile app is **teal on white, sans-serif**. The web client — which guests land on directly from the mobile invite, in the same flow, seconds apart — is **magenta on black with serif headings**. A guest's first impression of Duitlah is a visibly different product from the one their friend is holding at the table.

PRODUCT.md commits to differentiating through "confident, deliberate color" carrying "real surface area". Two brand colours across two clients works against that, particularly in the Bagilah moment the positioning depends on.

Not tested

- **OCR.** No `ANTHROPIC_API_KEY` configured, so receipt scans return an empty structured scan and fall through to manual review. Documented fallback behaviour, not a fault.
- **Camera capture.** The emulator's camera is a synthetic test image, so real capture quality is unassessed.

- **Bill finalization, Bagilah settlement generation, guest exclusion from settlements, finalized-history locking.** All unreachable behind B1.
- **Tax, service charge, discount and rounding-adjustment arithmetic.** Unreachable behind C1. The engine appears built; it could not be exercised.
- **Personal finance** — accounts, transactions, budgets, goals, insights. Out of scope for this pass.

Appendix — test method

The test bill was constructed so that a naive implementation would get it wrong: a shared item that does not divide into whole sen, two stacked charges, and a printed total one sen away from the computed total (Malaysian cash rounding).

Item	Qty	Unit	Line
Nasi Lemak Ayam	3	12.90	38.70
Teh Ais	2	3.50	7.00
Sotong Goreng	1	25.00	25.00

Subtotal 70.70 · service 10% = 7.07 · SST 6% on 77.77 = 4.67 · computed 82.44 · **receipt total 82.45** (rounding adjustment +0.01).

Claims: Haikal 2 nasi + ⅓ sotong; Aiman 1 nasi + 1 teh + ⅓ sotong; Siti 1 teh + ⅓ sotong.

The subtotal portion validated exactly (Pass 3). The charges and rounding portion could not be entered (C1), so the invariant `sum(final amounts) = declared receipt total` remains unverified — it is the single most valuable check to run once C1 is fixed.

The group test used RM 85.55 split two ways for the same reason: 42.775 forces a rounding decision.